

# Tenderfish UX, UI and Interaction Model

Developer brief for conversational intake, generated project preview, progressive dashboard logic, approvals, participant management and execution-facing UX.

|                                                                                                                                                            |                                                                                                                                                                                                             |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Core principle</b><br>Tenderfish should feel like a calm AI workspace that turns messy project communication into a visible, structured project system. | <b>Product behavior</b><br>Users should be able to talk to the bot, upload anything, forward communication, and immediately see a generated project preview with readiness, missing items and next actions. |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 1. Product behavior

Tenderfish is not only an intake tool and not only a chatbot. It should behave like a generative project system. The user provides messy project input through files, text, email, or messaging channels, talks to the bot, and immediately receives a visible project preview. That preview becomes the entry point into the full dashboard.

- The bot is a first class interface, not a support feature.
- The first result should be a project preview, not only a text summary.
- The preview should show structure, readiness, missing inputs and next actions.
- The dashboard should grow progressively by project stage and participant role.

## 2. Main UX principles

- **One calm intake surface** - a simple input area that accepts files, text, forwarded emails and bot conversation.
- **Input first, structure second** - the UI should visibly transform chaos into project structure.
- **Minimal surface, deep drilldown** - few top-level areas, detailed views only when needed.
- **Review instead of configuration** - the system should suggest structure and the user should mostly confirm, edit or reject.
- **Progressive reveal** - show the full path of the project, but only activate what is relevant now.

## 3. Supported input channels

All project information should flow into one shared intake engine. Different input channels must not create separate project truths.

| Channel               | UX intent                                                                               | Backend result                                                        |
|-----------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Direct upload         | Drop PDFs, drawings, spreadsheets, images, CSV files                                    | Raw input item with metadata and attachments                          |
| Paste text            | Fast way to add notes, briefings, copied email and text items                           | Raw text items, parsed and classified                                 |
| Conversation with bot | Primary interaction mode for asking the system for help, clarification, review, updates | Prioritized text and actions, project structure                       |
| Project email alias   | Forward emails or use CC/BCC for automatic project body                                 | Project body, header, sender and attachments parsed into object cards |
| Messaging gateway     | WhatsApp or Signal input, manually forwarded                                            | First, integrated text or file converted into candidates              |
| Manual quick entry    | Simple add note action for project team                                                 | Comment, issue candidate, task candidate or document note             |

## 4. Conversational interface

The user should be able to talk to the system in plain language. This must drive both analysis and interface state.

- Examples: *Show me the structure of this project*, *What is missing before tendering*, *Build the dashboard from this material*, *Prepare this for cost estimation*, *Which packages are still incomplete*.
- The bot should be able to explain what it understood, what is still uncertain, what needs approval and what the next best action is.
- The output of a conversation should not only be text. It should also update project objects, preview sections, readiness indicators and pending review items.

## 5. Structured Inbox

The Structured Inbox is the control layer between raw intake and confirmed project truth. It is the most important UX area in the system.

- **Queues** - New Inputs, Needs Review, Auto Classified, Low Confidence, Unassigned, Awaiting Role Confirmation, Awaiting Project Assignment.
- **Card content** - source, sender, timestamp, suggested type, suggested object link, confidence, next action.
- **Primary actions** - Approve and attach, Convert to document, Convert to issue, Convert to quote, Assign to package, Request clarification, Mark irrelevant.

## 6. Generated project preview

After the first prompt or first project upload, Tenderfish should generate a visible preview of the project system. This should feel like a first dashboard, not like a static summary.

- Show inferred project structure and main modules.
- Show what already exists and what is still missing.
- Show readiness toward cost estimate, tender package and execution setup.
- Show future modules calmly, even if they are not yet active.
- Require deeper registration only when the user wants to persist, edit, collaborate or approve.

## 7. Readiness logic

Readiness must be a core UI concept. The system should always tell the user where the project stands and what is missing to progress.

- Examples: Cost estimate readiness, Tender readiness, Award readiness, Execution setup readiness.
- Each readiness indicator should be linked to concrete missing inputs, unresolved reviews, required approvals and blocked next steps.
- Use simple visual components such as a progress bar, compact gauge, status card or meter. Keep it calm and highly legible.

## 8. Approval mechanics

Tenderfish must support formal approvals. This is not optional. The UI must surface explicit approve and release actions wherever project governance requires them.

- Architects or project leads must be able to review and formally release generated outputs.
- Typical approvals include cost structure approval, package approval, tender package release, participant role confirmation, quote acceptance, change order approval and selected execution confirmations.
- The UI should clearly show what is awaiting approval, who is responsible, what will happen after release and whether downstream modules unlock after approval.

| Approval object        | Typical action     | UI behavior                                           |
|------------------------|--------------------|-------------------------------------------------------|
| Generated structure    | Approve structure  | Large primary action in review panel                  |
| Tender package         | Release for tender | Confirmation step with audit event                    |
| Participant role       | Confirm role       | Quick action from participants view                   |
| Quote decision         | Approve award      | Decision action in comparison view                    |
| Change order           | Approve or reject  | Decision action with reason and trace                 |
| Execution confirmation | Sign off or accept | Mobile friendly confirmation with signature if needed |

## 9. Participant and access model

Not every contributor should need an account from day one. Tenderfish should separate contribution from system access.

- **External Source** - can send information but has no active project navigation.
- **Lightweight Participant** - invited for limited actions through magic links or narrow access.
- **Full Account User** - has role-based project access and can review, edit, approve or manage.
- The system may infer likely roles from communication, but hard permissions should only be granted after confirmation by an authorized project lead.

## 10. Role inference and role confirmation

- The system may infer likely roles from sender domain, document type, repeated behavior, company name, assigned package and conversation context.
- Possible suggestions include Architect, Internal Team, Vendor, Supplier, Client, Site User, Approver and Observer.
- Role inference should produce suggestions, not automatic permissions.
- Project admins should see a participant registry with quick actions such as Confirm role, Invite as lightweight participant, Invite as full user, Keep external, Restrict access and Assign to package.

## 11. Screen-by-screen information architecture

The product shell should remain minimal. The project should evolve through a small number of top-level areas.

| Screen            | Purpose                                 | Key modules                                                                     |
|-------------------|-----------------------------------------|---------------------------------------------------------------------------------|
| Project Home      | High-level orientation and next actions | Project header, readiness cards, pending reviews, recent activity, next actions |
| Structured Inbox  | Convert intake into project truth       | Queues, input cards, confidence, quick actions                                  |
| Structure         | Show generated project model            | Packages, cost groups, documents, timeline, participants snapshot               |
| Cost and Decision | Commercial and formal control           | Cost state, quotes, comparison, awards, approvals, change orders                |
| Execution         | Operational project usage               | Tasks, issues, site updates, photos, time entries, reports, approvals           |
| Participants      | Manage identities and access            | External sources, invited participants, active users, role proposals            |
| Settings          | Project configuration                   | Project aliases, messaging rules, email intake, permissions, integrations       |

## 12. Project Home screen

- Should feel calm and simple, similar in emotional clarity to Lovable, not like traditional construction software.
- Primary elements: project title, phase, current stage, readiness indicators, pending approvals, next actions and a large intake bar.
- The intake bar should allow upload, paste, email forwarding hints and bot interaction in one place.

## 13. Execution screen

- Must be mobile first in logic, even if early versions are browser based.
- Should present only a few obvious actions: My tasks, Upload photo, Log time, Add note, Report issue, Review approvals.
- Field users should never be exposed to unnecessary AVA or administrative complexity.

## 14. Progressive dashboard evolution

The same project dashboard should evolve with project maturity and participant complexity.

- Early phase: structure, documents, cost estimate readiness, missing inputs.
- Tendering phase: package completeness, formal release, quote intake, comparison, award decisions.
- Execution phase: site updates, reports, issues, documentation, signatures, change orders.
- Later stage modules may already be visible as part of the lifecycle, but should remain quiet until activated.

## 15. Minimalist UI direction

- Do not make Tenderfish look like classic AVA software, ERP software or a file system.
- Use cards, status bands, simple tabs and clear next actions instead of dense menus and heavy tables on the first level.
- Hide complexity until it becomes useful.
- Prefer one calm input surface, one structured inbox and one evolving project surface.
- Tables should be used for comparison or detail views only where they genuinely help.

## 16. Developer implications

- The backend should not only classify data. It should also generate interface state: visible modules, hidden modules, readiness values, pending reviews, suggested next actions and approval requirements.
- Every execution event must attach back to existing project objects. The Execution Layer must not create a second project truth.
- Approvals must be first-class objects with status, responsible role, timestamp and audit trace.
- Email and messaging channels are feeder channels into the same intake engine, not separate systems.

## 17. Suggested MVP sequence

- Stage 1: conversational intake, file upload, project email alias, structured inbox, generated project preview.
- Stage 2: readiness indicators, participant registry, role suggestions, manual confirmations, basic approvals.
- Stage 3: cost and decision views, quote comparison, package release, tender readiness logic.
- Stage 4: execution objects, mobile-friendly updates, issues, photos, time entries, site notes and sign-off flows.

## 18. One-sentence internal definition

Tenderfish should behave like a generative project system that turns messy project communication into a structured, reviewable, approvable and execution-ready dashboard.